UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Diseño de un Agente Inteligente — Documento ajustado

CREDITSCORE-AGENT — Agente inteligente de evaluación de riesgo crediticio

Versión 2: incluye selección justificada, desarrollo del agente y pruebas de desempeño

Presentado por:

Steven David Contreras — T00077010

Andres Camilo Castaño — T00077278

Curso: Inteligencia Artificial

Docente: Edwin Alexander Puertas

Cartagena, 17/10/2026

Tabla de cambios respecto de la entrega anterior (Diseño inicial, 27/08/2026)

Cada fila indica la sección afectada, lo que decía el documento anterior, lo que dice ahora y el motivo. Los cambios responden a la retroalimentación del docente (texto con marcas de herramientas de citación, encabezados que no correspondían al contenido y ausencia de verificación) y a los nuevos requisitos de la actividad.

Sección	Contenido anterior	Contenido nuevo	Justificación
Portada	Un solo integrante (Steven David Contreras).	Dos integrantes de la pareja.	La entrega es por parejas.
Todo el documento	Texto con marcas literales de herramientas de búsqueda y citación (por ejemplo «cite,turn0search4», «filecite turn0file1 L22-L24») en al menos cinco secciones.	Marcas eliminadas; el texto se redactó sin ellas y las fuentes se citan por autor y año en Referencias.	Retroalimentación del docente: eran artefactos no revisados.
Secciones 4 a 7	Encabezados desordenados: las entradas aparecían bajo «Descripción del entorno», las salidas bajo «Entradas y sensores», el texto SMART bajo «Salidas y actuadores» y «Medidas de desempeño» estaba vacío.	Cada encabezado contiene su contenido: 4 entorno; 5 entradas y sensores (Tabla 1); 6 salidas y actuadores (Tabla 2); 7 medidas de desempeño.	Retroalimentación del docente: contenido pegado sin reordenar.
7. Medidas de desempeño	Dos metas SMART (AUC-ROC ≥ 0,75 y reducción del 20 % del tiempo) y métricas complementarias sin umbral.	Se conservan las metas y se agregan fórmulas, umbrales fijados antes de ejecutar y la reformulación de la medida de tiempo (sección 9.7).	La reducción del 20 % se probó con una línea base manual (sección 9.7).
Sección A (nueva)	No existía.	Selección justificada del agente: matriz ponderada, cálculo, sensibilidad y justificación.	Requisito de la Fase 1 de la actividad.
9. Desarrollo del agente (nueva)	No existía; las conclusiones eran la sección 9.	Nueve subsecciones: arquitectura, tecnologías, entradas, salidas, UML, diferencias diseño–implementación, pruebas, limitaciones y enlaces.	Requisito de la Fase 2. Las conclusiones pasan a ser la sección 10.
10. Conclusiones	Concluía viabilidad técnica sin evidencia de ejecución.	Conclusiones actualizadas con resultados reales y sus límites (datos sintéticos).	Coherencia con las pruebas ejecutadas.
Referencias	Russell y Norvig (2021) aparecía dos veces; sin verificación de enlaces.	Referencia duplicada fusionada; se agregan las fuentes técnicas usadas. Enlaces marcados para verificación.	Calidad de las referencias.

A. Selección justificada del agente (sección nueva)

La actividad pide implementar uno de los dos agentes diseñados en la entrega anterior: CREDITSCORE-AGENT (riesgo crediticio, diseño de Steven David Contreras) y Cartrescue-agent (recuperación de carritos abandonados, diseño del compañero de pareja). Los criterios y pesos se definieron antes de puntuar, con los valores sugeridos por la actividad.

Fuente de los puntajes del agente B. El documento de diseño completo de Cartrescue-agent no estaba disponible al elaborar esta versión. Sus puntajes se sustentan en la descripción oficial del caso (Intelligent Agents – Use Cases, p. 3) y en las características del conjunto público REES46, no en el diseño del compañero; por eso se marcan como «caso de uso». Los del agente A se apoyan en su diseño y en el prototipo construido, lo que puede sesgar la comparación a favor de A. Los integrantes deben contrastar las filas de B con su diseño real y, si cambian, volver a ejecutar «python scripts/matriz_decision.py» después de editar docs/matriz_decision.json.

Tabla A1. Matriz de decisión ponderada (A = CREDITSCORE-AGENT; B = Cartrescue-agent, puntajes según el caso de uso)

Criterio	Peso (w)	Agente A: puntaje (1–5)	Agente A: evidencia	Agente B: puntaje (1–5)	Agente B: evidencia
Viabilidad técnica	25 %	4	Modelo logístico, reglas y búsqueda de casos similares con solo numpy; sin GPU ni APIs. El prototipo de esta entrega corre y pasa 20 pruebas unitarias. Se resta 1 porque la integración real con originación y buró no es posible en el curso (se simula).	4	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). El caso describe percepción de eventos de carrito, un modelo de probabilidad de abandono, una decisión de intervención según el desempeño histórico y actuadores de correo o descuentos. Cinco de esos seis componentes (percepción simulada, modelo de probabilidad, decisión, actuadores simulados y registro de resultados con aprendizaje) ya se implementaron en el otro agente; falta la captura en tiempo real en el navegador (movimiento del ratón).
Disponibilidad de datos o entorno	20 %	3	No existen las 5.000 solicitudes históricas reales; se construyó un simulador sintético propio (src/creditscore/simulador.py). El simulador existe, pero la señal predictiva la define el propio simulador.	4	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). El caso recomienda usar el clickstream de la tienda o, alternativamente, el conjunto público REES46 (más de 30 millones de eventos). REES46 trae eventos view, cart, remove_from_cart y purchase por sesión, de los que se puede derivar el abandono; no trae movimientos del ratón ni respuestas a intervenciones, que habría que simular.
Medibilidad del desempeño	20 %	4	El diseño define AUC-ROC ≥ 0,75 (fórmula y umbral explícitos), medible con salidas del programa. La reducción del 20 % del tiempo se probó con una línea base manual simulada (3,6 s sobre 10 solicitudes), sin digitación ni consulta real al buró.	3	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). La meta del caso (recuperar al menos 15 % más de carritos que la tasa actual durante tres semanas) exige una tienda piloto, una tasa base real y un grupo de comparación; con datos simulados el efecto lo fijaría el propio simulador. Sí se pueden medir sin tienda el AUC del modelo de abandono y la latencia.
Complejidad de IA	15 %	5	Agente de aprendizaje (recalibración ante deriva) con decisión por utilidad esperada y abstención por incertidumbre; cubre los siete componentes IBM (ver sección 9.1).	4	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). Un modelo estima la probabilidad de abandono y la intervención (recordatorio, descuento o producto alternativo) se elige según el perfil y el desempeño histórico por tipo, registrando el resultado para refinar la estrategia: agente de aprendizaje con decisión por utilidad. No llega a 5 porque el caso no menciona detección de deriva ni abstención por incertidumbre.
Costo y recursos	10 %	5	Costo de 0 COP en APIs; Python y numpy gratuitos; se ejecuta en un portátil (latencia p95 inferior a 1 ms medida en logs/resultados_pruebas.json).	3	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). El caso exige una tienda piloto durante tres semanas, un servicio de correo o notificaciones push y un motor de descuentos. Pueden simularse sin costo, pero una prueba real requeriría permisos y servicios con cuota.

Criterio	Peso (w)	Agente A: puntaje (1–5)	Agente A: evidencia	Agente B: puntaje (1–5)	Agente B: evidencia
Valor del problema	10 %	4	Usuario identificable (analista de crédito) y beneficio concreto (priorizar y dar consistencia a la evaluación). El beneficio real no se verificó con datos reales.	4	Caso de uso de Cartrescue (Intelligent Agents – Use Cases, p. 3). Usuario identificable (la tienda en línea) y beneficio concreto (carritos recuperados, es decir, ventas); el caso cita la fuente de diseño de Datagrid sobre el ROI del abandono de carritos. El beneficio no se verificó con datos reales.
Puntaje ponderado S	**100 %**	**4,05**		**3,70**	

A.1 Cálculo del puntaje ponderado

Se aplica $S(\text{agente}) = \Sigma\,(w_i \times p_i)$, con $\Sigma\, w_i = 1$ y puntaje máximo 5.

```
S(A) = 0,25×4 + 0,20×3 + 0,20×4 + 0,15×5 + 0,10×5 + 0,10×4 = 4,05
S(B) = 0,25×4 + 0,20×4 + 0,20×3 + 0,15×4 + 0,10×3 + 0,10×4 = 3,70
```

A.2 Análisis de sensibilidad

El criterio más influyente se definió como el que más aporta a la diferencia entre agentes, es decir, el de mayor $w_i \cdot |pA - pB|$. Hay un empate: «Disponibilidad de datos o entorno», «Medibilidad del desempeño», «Costo y recursos» aportan 0,20 puntos cada uno. Como el empate impide elegir uno solo, se probó cada uno de ellos con +10 y −10 puntos porcentuales de peso, reescalando los demás proporcionalmente para sumar 100 %.

Tabla A2. Sensibilidad de la decisión

Escenario	Pesos en % (viabilidad / datos / medibilidad / complejidad / costo / valor)	S(A)	S(B)	Elegido
Original	25,0 / 20,0 / 20,0 / 15,0 / 10,0 / 10,0	4,050	3,700	A
Disponibilidad de datos o entorno +10 pp	21,9 / 30,0 / 17,5 / 13,1 / 8,8 / 8,8	3,919	3,737	A
Disponibilidad de datos o entorno -10 pp	28,1 / 10,0 / 22,5 / 16,9 / 11,2 / 11,2	4,181	3,663	A
Medibilidad del desempeño +10 pp	21,9 / 17,5 / 30,0 / 13,1 / 8,8 / 8,8	4,044	3,612	A
Medibilidad del desempeño -10 pp	28,1 / 22,5 / 10,0 / 16,9 / 11,2 / 11,2	4,056	3,788	A
Costo y recursos +10 pp	22,2 / 17,8 / 17,8 / 13,3 / 20,0 / 8,9	4,156	3,622	A
Costo y recursos -10 pp	27,8 / 22,2 / 22,2 / 16,7 / 0,0 / 11,1	3,944	3,778	A
Pesos iguales (1/6)	16,7 / 16,7 / 16,7 / 16,7 / 16,7 / 16,7	4,167	3,667	A
Robustez: puntajes de B + 1 en todos los criterios (tope 5), pesos originales	25 / 20 / 20 / 15 / 10 / 10	4,050	4,700	B

Resultado: con los puntajes actuales la decisión se mantiene en A en los 8 escenarios de pesos (original, +10 pp y −10 pp de cada criterio empatado, y pesos iguales). La ventaja mínima de A es de 0,17 puntos (escenario «Costo y recursos -10 pp»). En cambio, la decisión no es robusta a los puntajes de B: la ventaja de A con los pesos originales es de 0,35 puntos, que equivale a un punto de diferencia en criterios que sumen un 35 % del peso (por ejemplo, viabilidad técnica y costo). Si el diseño real de Cartrescue-agent justificara un punto más en todos los criterios, B obtendría 4,70 frente a 4,05 y pasaría a ser la mejor opción. La ventaja de A debe confirmarse con el diseño real de B.

A.3 Justificación de la selección (una página)

Se selecciona CREDITSCORE-AGENT con un puntaje ponderado de 4,05 sobre 5, frente a 3,70 para Cartrescue-agent. A supera a B en medibilidad del desempeño (0,20), complejidad de ia (0,15), costo y recursos (0,20): la meta de A puede medirse sin tienda piloto, funciona con Python y numpy sin APIs ni servicios de pago, y es un agente de aprendizaje con decisión por utilidad y abstención por incertidumbre.

B supera a A en disponibilidad de datos o entorno (0,20), porque cuenta con un conjunto público real (REES46). Viabilidad técnica y valor del problema empatan.

Resultado de la sensibilidad: al subir o bajar 10 puntos el peso de cada criterio empatado, A conserva el primer lugar en los seis casos (la menor ventaja es 0,17 puntos). Con pesos iguales A también gana (4,17 frente a 3,67). La decisión depende de los puntajes de B, como se indicó en A.2.

Ventajas de la elección: medibilidad directa mediante AUC-ROC, calibración, trazabilidad y latencia; una arquitectura que cubre los siete componentes IBM; y ejecución completa en un portátil, sin costo. Limitaciones: no hay datos reales de solicitudes, por lo que se usa un simulador sintético y los resultados demuestran el funcionamiento de la tubería, no el desempeño en una entidad financiera; la reducción del 20 % del tiempo se probó con una línea base manual simulada (ver 9.7); y la evaluación de equidad entre subgrupos no es posible con estos datos.

Qué se pierde al descartar a Cartrescue-agent: un problema con retroalimentación inmediata (el cliente compra o no tras el recordatorio), que permitiría medir el aprendizaje en ciclos cortos; un conjunto de datos público y real (REES46); y un dominio con menor riesgo ético que las decisiones crediticias. La descripción del caso confirma que Cartrescue-agent también aprende de los resultados y dispone de datos, y aun así la decisión se mantiene en A por medibilidad y costo; debe revisarse solo si el diseño completo de B justifica puntajes más altos en esos criterios.

1. Introducción y planteamiento del problema

El proceso de evaluación crediticia es una actividad crítica para las entidades financieras, porque requiere estimar la probabilidad de incumplimiento de las obligaciones adquiridas por un solicitante. En un procedimiento predominantemente manual, el análisis de información financiera, historial de pagos, nivel de endeudamiento y líneas de crédito abiertas puede demandar tiempo considerable y presentar variaciones derivadas de la carga operativa, los criterios aplicados y la disponibilidad de información. El diseño de un agente inteligente permite automatizar parte del proceso de percepción, análisis y recomendación, manteniendo supervisión humana para los casos con incertidumbre o características atípicas.

El CREDITSCORE-AGENT evalúa el riesgo de incumplimiento de nuevas solicitudes de crédito. Según el caso de uso del catálogo del curso, el objetivo es alcanzar un AUC-ROC mínimo de 0,75 en la predicción de incumplimiento y reducir en 20 % el tiempo de respuesta frente al proceso manual, usando una muestra de 5.000 solicitudes históricas. El agente recibe variables financieras del solicitante, estima una probabilidad de incumplimiento y una calificación de riesgo, contrasta el resultado con decisiones históricas sobre solicitudes similares y registra la evaluación en el sistema de originación. El resultado final de cada solicitud se conserva para recalibrar periódicamente el modelo (Curso de Inteligencia Artificial, 2026).

2. Tipo de agente inteligente

Según la clasificación de Russell y Norvig (2021), el CREDITSCORE-AGENT es principalmente un agente de aprendizaje, con características complementarias de un agente basado en utilidad. Es de aprendizaje porque usa resultados históricos para construir y actualizar el modelo de predicción: cada solicitud tiene variables de entrada y un resultado observado posteriormente, lo que permite evaluar las predicciones y recalibrar. Es también basado en utilidad porque la decisión no es una condición binaria: pondera el riesgo estimado, la incertidumbre, los criterios de la política crediticia y la conveniencia de remitir casos a revisión humana.

3. Tipos de ambiente

Propiedad	Clasificación	Justificación
Observabilidad	Parcialmente observable	Solo se dispone de las variables suministradas y del historial; no se observan los factores futuros que afectan la capacidad de pago.
Determinismo	Estocástico	La relación entre las características y el incumplimiento es probabilística.

Propiedad	Clasificación	Justificación
Episodicidad	Secuencial	Cada solicitud es una unidad de evaluación, pero las decisiones previas y sus resultados influyen en la recalibración y en evaluaciones futuras.
Dinamismo	Dinámico	Cambian las condiciones económicas, las políticas y la composición de los datos; el agente debe recalibrarse.
Discretización	Principalmente continuo	Ingresos, relación deuda-ingreso y probabilidades son continuos; las acciones (aprobar, rechazar, remitir) son discretas.
Número de agentes	Agente único con humanos y sistemas externos	El núcleo es un agente que interactúa con analistas y con el sistema de originación.

4. Descripción del entorno

El entorno está compuesto por la entidad financiera, los solicitantes, los analistas de revisión, el sistema de originación, las fuentes de información financiera y la infraestructura de datos para entrenamiento, validación y seguimiento del modelo. El agente se ubica entre la recepción de una solicitud y la generación de una recomendación de riesgo.

En condiciones normales, el sistema recibe una solicitud con información suficiente, valida la integridad de los datos, ejecuta el modelo, genera una probabilidad de incumplimiento y asigna una categoría de riesgo. Después registra el resultado y puede recomendar aprobación, rechazo o revisión humana según los umbrales y políticas de la organización. En condiciones excepcionales (datos incompletos, valores inconsistentes, resultados fuera de la distribución histórica o incertidumbre elevada) el agente debe abstenerse de producir una decisión automática y remitir el caso a revisión especializada.

5. Entradas y sensores

Los sensores del agente son componentes de software que capturan, validan y transforman las señales del entorno: API, conectores con bases de datos, formularios digitales, módulos de validación y procesos de preprocesamiento. Su función es convertir las percepciones en una representación estructurada apta para el modelo predictivo. El caso de uso contempla ingresos, historial de pagos, relación deuda-ingreso, líneas de crédito abiertas y el historial de decisiones sobre solicitudes similares.

Tabla 1. Entradas, tipos de datos y mecanismos de captura

Entrada	Tipo de dato	Captura
Ingresos	Numérico continuo	Formulario, sistema financiero o API de verificación
Historial de pagos	Datos históricos tabulares; variables categóricas y temporales	Base de datos o API de información crediticia
Relación deuda-ingreso	Numérico continuo	Cálculo automático a partir de ingresos y obligaciones registradas
Líneas de crédito abiertas	Numérico discreto y atributos tabulares	Base de datos o API de información crediticia
Historial de decisiones	Tabular y categórico	Base de datos del sistema de originación
Resultado observado del crédito	Categórico binario y temporal	Sistema de seguimiento de cartera

6. Salidas y actuadores

Las salidas son las acciones o recomendaciones del agente. Los actuadores son los mecanismos mediante los cuales modifican el estado del sistema de originación o activan la intervención humana.

Tabla 2. Salidas, actuadores y efecto sobre el entorno

Salida	Actuador	Efecto sobre el entorno
Aprobación recomendada	API del sistema de originación	Registra la recomendación y permite continuar el flujo de aprobación según la política institucional.
Rechazo recomendado	API del sistema de originación	Registra el resultado y detiene o deriva el flujo de formalización del crédito según las reglas vigentes.

Salida	Actuador	Efecto sobre el entorno
Remisión a revisión humana	Módulo de notificaciones y bandeja de analistas	Asigna el caso a un responsable humano para una evaluación adicional.
Calificación de riesgo	Módulo de scoring y registro	Incorpora la puntuación al expediente crediticio y permite aplicar los umbrales definidos.
Registro histórico	Base de datos de auditoría y aprendizaje	Conserva entradas, salida, versión del modelo y resultado posterior para trazabilidad y recalibración.

7. Medidas de desempeño (SMART)

- **Específica:** evaluar automáticamente el riesgo de incumplimiento de solicitudes de crédito a partir de variables financieras e históricas y generar una recomendación de riesgo.
- **Medible:** alcanzar un AUC-ROC mínimo de 0,75 en la predicción de incumplimiento y reducir en 20 % el tiempo de respuesta respecto del proceso manual.
- **Alcanzable:** validar el desempeño con una muestra de 5.000 solicitudes históricas, como establece el caso de uso.
- **Relevante:** mejorar la rapidez y consistencia de la evaluación, dando apoyo analítico a la decisión crediticia.
- **Temporal:** comprobar los indicadores durante una fase piloto definida por la organización y antes de una eventual puesta en producción.

Como métricas complementarias se recomienda monitorizar precisión, recall de la clase de incumplimiento, calibración de probabilidades, tasa de falsos positivos, estabilidad temporal y diferencias de desempeño entre subgrupos. En la sección 9.7 cada medida se convierte en una prueba con fórmula, umbral previo y resultado numérico; la reducción del 20 % se prueba con una línea base manual.

8. Componentes de la arquitectura de IA agéntica (marco IBM)

IBM (2025) describe los agentes de IA como sistemas con componentes interconectados que permiten percibir, procesar, razonar, actuar, comunicarse y aprender. La plantilla organiza el diseño en siete componentes: percepción y manejo de entradas; planificación y descomposición de tareas; memoria; razonamiento y toma de decisiones; llamada de herramientas y acción; comunicación; y aprendizaje y adaptación.

Percepción y manejo de entradas. Recibe la solicitud y transforma los datos en una representación estructurada: validación de campos, detección de valores ausentes o inconsistentes, normalización y transformaciones del modelo, además de verificar que la solicitud esté dentro del dominio esperado. Una interpretación incorrecta de los datos afecta las decisiones posteriores.

Planificación y descomposición de tareas. Descompone la evaluación en: recibir la solicitud, validar la información, consultar los datos, ejecutar la predicción, contrastar con casos similares, verificar reglas, determinar la necesidad de revisión humana y registrar la salida.

Memoria. La memoria de corto plazo conserva el contexto de la solicitud en evaluación (variables, transformaciones, resultado del modelo, reglas activadas y acciones). La de largo plazo almacena historial de solicitudes, resultados observados, decisiones, versiones del modelo y métricas, con controles de acceso.

Razonamiento y toma de decisiones. Combina el resultado probabilístico con las políticas y umbrales de la entidad, la calidad de los datos, la similitud con casos históricos y la incertidumbre, para decidir si la solicitud sigue un flujo automatizado o se remite a un analista. No sustituye la gobernanza institucional.

Llamada de herramientas y acción. Usa herramientas de software (por API o funciones estructuradas) para consultar fuentes de datos, ejecutar el modelo, obtener información del sistema de originación y registrar el resultado.

Comunicación. Intercambia información con sistemas institucionales y analistas. Para el analista produce una salida estructurada con calificación, probabilidad, factores relevantes y razón de la remisión; para los sistemas, interfaces estructuradas.

Aprendizaje y adaptación. Usa los resultados observados como retroalimentación: compara probabilidades estimadas con resultados reales, detecta cambios en la distribución de los datos y recalibra el modelo cuando se cumplen los criterios establecidos.

9. Desarrollo del agente

Esta sección describe el agente implementado en el repositorio (carpeta creditscore_agent). Todas las cifras proceden de ejecuciones reales registradas en la carpeta logs/. Los datos de solicitudes son sintéticos, generados por un simulador propio.

9.1 Arquitectura

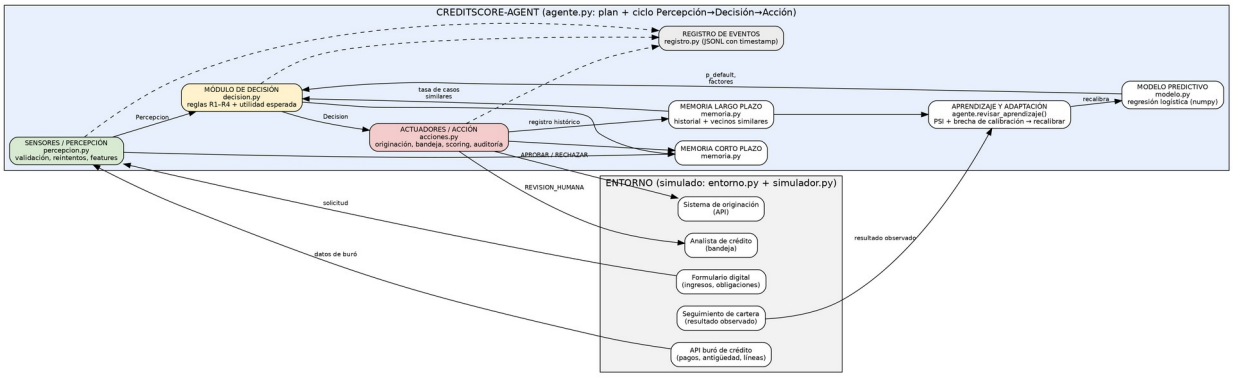


Figura 1. Diagrama de componentes del CREDITSCORE-AGENT (fuente: docs/diagramas/arquitectura.dot)

El ciclo Percepción → Decisión → Acción está en AgenteCreditScore.evaluar(). Los sensores (formulario y buró) entregan datos al módulo de percepción, que valida y construye el vector de características. El módulo de decisión consulta el modelo predictivo y la memoria de largo plazo (casos similares), aplica las reglas y la función de utilidad, y entrega una Decision. Los actuadores ejecutan esa decisión sobre el entorno (originación, bandeja de analistas, expediente y auditoría) y el registro de eventos deja constancia con marca de tiempo. El resultado observado del crédito llega después desde la cartera y alimenta la revisión de aprendizaje.

Tabla 3. Correspondencia entre componentes y código

Componente de la arquitectura	Archivo / clase	Qué hace
Sensores y percepción	percepcion.py · ModuloPercepcion	Lee formulario y buró (con reintentos), valida tipos y rangos, calcula el DTI y construye 5 características.
Modelo predictivo	modelo.py · ModeloRiesgo	Regresión logística con regularización L2 (Newton-Raphson ponderado); contribuciones por variable para explicar.
Estado interno (memoria)	memoria.py · MemoriaCortoPlazo, MemoriaLargoPlazo	Contexto de la solicitud; historial con resultados y búsqueda de los 25 casos más similares.
Módulo de decisión	decision.py · ModuloDecision	Reglas R1 a R4 y minimización del costo esperado entre aprobar, rechazar y remitir.
Actuadores y acción	acciones.py · Actuadores	Originación con reintentos y outbox, bandeja de analistas, expediente de calificación y auditoría.
Planificación y ciclo	agente.py · AgenteCreditScore	Plan de ocho tareas y orquestación; manejo de errores con decisión segura (revisión humana).
Aprendizaje y adaptación	agente.py · revisar_aprendizaje()	Detecta deriva (PSI y brecha de calibración) y recalibra el modelo con ponderación de casos recientes.
Entorno simulado	entorno.py, simulador.py, fabrica.py	Genera solicitudes sintéticas, simula buró y originación con fallos inyectables y entrega el resultado observado.
Registro de eventos	registro.py	Escribe JSON Lines con marca de tiempo de cada percepción, decisión, acción y fallo.

Relación con los siete componentes IBM (los siete se verificaron en la sección 8 de este mismo documento):

Tabla 4. Componentes IBM y su implementación

Componente IBM (sección 8)	Implementación en el código	Estado	Limitación
1. Percepción y manejo de entradas	percepcion.py	Implementado	Sensores simulados; el historial de pagos se reduce a dos variables numéricas.
2. Planificación y descomposición de tareas	AgenteCreditScore.PLAN y evaluar()	Parcial	Secuencia fija de ocho tareas registrada en el log; no hay replanificación dinámica.
3. Memoria	memoria.py	Implementado	En memoria RAM; guardar() persiste a archivo, pero no se carga automáticamente al iniciar.
4. Razonamiento y toma de decisiones	decision.py	Implementado	Costos de decisión definidos por los autores, no por una política institucional real.
5. Llamada de herramientas y acción	acciones.py y entorno.py	Parcial	Las «herramientas» son funciones de un entorno simulado; no hay APIs reales ni modelo de lenguaje.
6. Comunicación	Payload a analista y originación; registro.py	Parcial	Salida estructurada con razón y tres factores principales; sin explicación en lenguaje natural.
7. Aprendizaje y adaptación	modelo.py y revisar_aprendizaje()	Implementado	La revisión se invoca manualmente; no hay programación periódica.

Ningún componente quedó sin implementar; los marcados como parciales tienen la limitación indicada. No se usa ningún modelo de lenguaje: todo el razonamiento lo realiza el código del repositorio.

9.2 Tecnologías

Tabla 5. Tecnologías usadas y evaluadas

Tecnología	Versión	Función	Justificación	Alternativa considerada	Motivo de la elección	Uso
Python	3.12.3	Lenguaje del agente	Permite el ciclo completo con poco código	Java	Mayor familiaridad y ecosistema científico	Usada
NumPy	2.4.4	Álgebra y regresión logística	Única dependencia del agente	scikit-learn 1.8.0	Evitar dependencia y mostrar el mecanismo de aprendizaje (Newton-Raphson)	Usada
unittest (biblioteca estándar)	Python 3.12.3	Pruebas automatizadas	No requiere instalación	pytest	pytest no estaba disponible sin red; las pruebas también corren con pytest	Usada
Graphviz	2.43.0	Diagrama de componentes	Reproducible desde .dot	draw.io	Fuente en texto versionable	Usada
Mermaid (fuentes .mmd) y render_secuencia.py con Matplotlib 3.10.8	—	Diagramas de secuencia	Fuente en texto; PNG generado por el mismo archivo	PlantUML, mermaid-cli	mermaid-cli no pudo ejecutarse (sin navegador); se escribió un renderizador propio	Usada
python-docx / LibreOffice	1.2.0 / 24.2.7.2	Generación del informe DOCX y PDF	Informe reproducible desde datos reales	Edición manual	Evita transcribir cifras a mano	Usada
scikit-learn	1.8.0	Modelos de clasificación	Biblioteca estándar de ML	—	Descartada para mantener una sola dependencia	Solo evaluada
Streamlit y pandas (app.py)	1.66.0 (probada)	Interfaz web y despliegue público (Opción A)	Evalúa solicitudes y ejecuta las pruebas en vivo; despliegue gratuito desde GitHub	Video (Opción B)	Los integrantes no pueden grabar juntos; la interfaz no contiene lógica de decisión	Usada

9.3 Entradas y sensores

Tabla 6. Entradas del agente (config/config.json define los rangos)

Entrada	Tipo	Rango o formato	Origen	Validaciones	Ejemplo
id_solicitud	Texto	No vacío; existente en el entorno	Interfaz / originación	Si no existe: INVALIDA y revisión humana	E01
ingresos_mensuales	Decimal (COP)	[500.000; 50.000.000]	Formulario (simulado)	Numérico finito; faltante: INCOMPLETA; negativo o no numérico: INVALIDA; fuera de rango: FUERA_DOMINIO	8.000.000

Entrada	Tipo	Rango o formato	Origen	Validaciones	Ejemplo
obligaciones_mensuales	Decimal (COP)	[0; 100.000.000]	Formulario (simulado)	Igual que ingresos	800.000
dti (derivada)	Decimal	[0; 3] (obligaciones / ingresos)	Calculada por el agente	Fuera de rango: FUERA_DOMINIO (inconsistente)	0,10
pagos_atrasados_24m	Entero	[0; 24]	Buró de crédito (simulado)	Debe ser entero; fuera de rango: FUERA_DOMINIO; 3 reintentos si el buró falla	0
antiguedad_historial_meses	Entero	[0; 600]	Buró de crédito (simulado)	Igual que pagos atrasados	120
lineas_credito_abiertas	Entero	[0; 30]	Buró de crédito (simulado)	Igual que pagos atrasados	3
Casos históricos similares	Vector de 5 variables y resultado 0/1	Mínimo 25 casos con resultado	Memoria de largo plazo	Si hay menos de 25, no se aplica la regla R3	3.500 casos de entrenamiento
resultado_observado	Entero 0/1	0 = pagó; 1 = incumplió	Cartera (simulada)	Solo se asocia a solicitudes evaluadas	1

9.4 Salidas y acciones

Tabla 7. Salidas y actuadores

Salida / acción	Formato	Módulo	Efecto esperado	Ejemplo real (logs/escenarios_eventos.jsonl)
Aprobación o rechazo recomendado	Diccionario: id, recomendacion, calificacion, p_default, version_modelo	acciones.py · _originacion()	Se registra en el sistema de originación; con 3 fallos queda DIFERIDA en el outbox	E01: APROBAR, p = 0,0155, calificación A, REGISTRADA
Remisión a revisión humana	Diccionario: id, calificacion, p_default, razon, factores, reglas	acciones.py · ejecutar()	El caso entra en la bandeja del analista con la razón de la abstención	E05: REVISION_HUMANA por R4 (DTI 0,8001)
Calificación de riesgo	Letra A–E por bandas de probabilidad (A < 5 %; B < 10 %; C < 20 %; D < 35 %; E ≥ 35 %)	decision.py · calificar()	Se registra en el expediente	p = 0,0155 → A
Registro histórico	Línea JSON de auditoría con entradas, versión, probabilidad, acción y razón	acciones.py · ejecutar(); registro.py	Trazabilidad y datos para recalibración	E14: {"accion": "APROBAR", "calificacion": "A", "estado_acciones": {"calificacion": "REGISTRADA", "originacion": "DIFERIDA", "historico": "REGISTRADO"}}

9.5 Diagramas de secuencia UML

Los diagramas se escribieron en Mermaid (archivos .mmd) y se renderizaron con docs/diagramas/render_secuencia.py. Los mensajes siguen las llamadas reales del código; los fragmentos loop y alt corresponden a los ciclos de reintento y a las ramas de decisión. Las Figuras 2 a 4 aparecen en las páginas horizontales que siguen a los extractos de código.

Explicación de la Figura 2. La interfaz invoca evaluar(id). La percepción lee el formulario y consulta el buró dentro de un ciclo de hasta tres intentos (mensajes 4 a 7). Con una percepción válida, la decisión obtiene la probabilidad y los factores del modelo (10 y 11) y la tasa de los casos similares (12 y 13). El fragmento alt de los mensajes 14 y 15 refleja el orden de las reglas: R4 (límite de DTI) y R3 (desacuerdo con casos similares) remiten a revisión; si no se activan, R2 elige la acción de menor costo esperado. Los actuadores registran la calificación, ejecutan la acción principal (alt de los mensajes 18 a 20) y registran auditoría y memoria (21). En el código, el evento AUDITORIA se escribe dentro de Actuadores.ejecutar(), antes del evento ACCION que emite el agente.

Explicación de las Figuras 3 y 4. Con el buró caído, la percepción agota tres intentos, registra tres eventos SENSOR_FALLO y devuelve un estado INCOMPLETA; la regla R1 hace que el agente se abstenga y remita el caso, sin llamar a originación. Con la originación caída, la decisión sigue siendo

APROBAR, pero tras tres intentos fallidos la recomendación se guarda en el outbox (estado DIFERIDA) y se audita, de modo que no se pierde. Ambos comportamientos están cubiertos por los escenarios E12 y E14 de las pruebas.

Correspondencia con el código real (extractos del repositorio):

```
per = None
try:
    per = self.percibir(id_sol)
    dec = self.decidir(per)
except Exception as e:   # fail-safe: ante error interno nunca se decide automaticamente
    self.registro.evento("ERROR_INTERNO", id_sol, error=repr(e))
    per = per or Percepcion(id_sol, {}, INVALIDA, ["error_interno"])
    dec = Decision(REVISION, None, "N/D", reglas=["R0_ERROR_INTERNO"], razon=f"error interno: {e!r}")
estados = self.actuar(dec, per)            # ACCION
```

```
def _leer_buro(self, id_sol):
    for intento in range(1, self.reintentos + 1):
        try:
            return self.entorno.consultar_buro(id_sol)
        except BuroNoDisponible as e:
            self.registro.evento("SENSOR_FALLO", id_sol, sensor="buro", intento=intento, error=str(e))
    return None
```

```
# R4: politica crediticia (limite de endeudamiento definido por la entidad)
if percepcion.dti is not None and percepcion.dti > self.c["dti_politica_max"]:
    return Decision(REVISION, reglas=["R4_POLITICA_DTI"],
                    razon=f"DTI {percepcion.dti:.2f} supera el limite de politica
{self.c['dti_politica_max']}",
                    **base)
# R3: desacuerdo modelo vs. casos historicos similares -> incertidumbre alta
if desacuerdo is not None and desacuerdo > self.c["desacuerdo_max"]:
    return Decision(REVISION, reglas=["R3_INCERTIDUMBRE"],
                    razon=f"Modelo ({p:.2f}) y casos similares ({p_vec:.2f}) discrepan en {desacuerdo:.2f}",
                    **base)
```

```
def _originacion(self, id_sol, payload):
    for intento in range(1, self.reintentos + 1):
        try:
            self.entorno.registrar_originacion(id_sol, payload)
            return "REGISTRADA"
        except OriginacionNoDisponible as e:
            self.registro.evento("ACTUADOR_FALLO", id_sol, actuador="originacion",
                                 intento=intento, error=str(e))
    self.pendientes.append(payload)
    return "DIFERIDA"
```

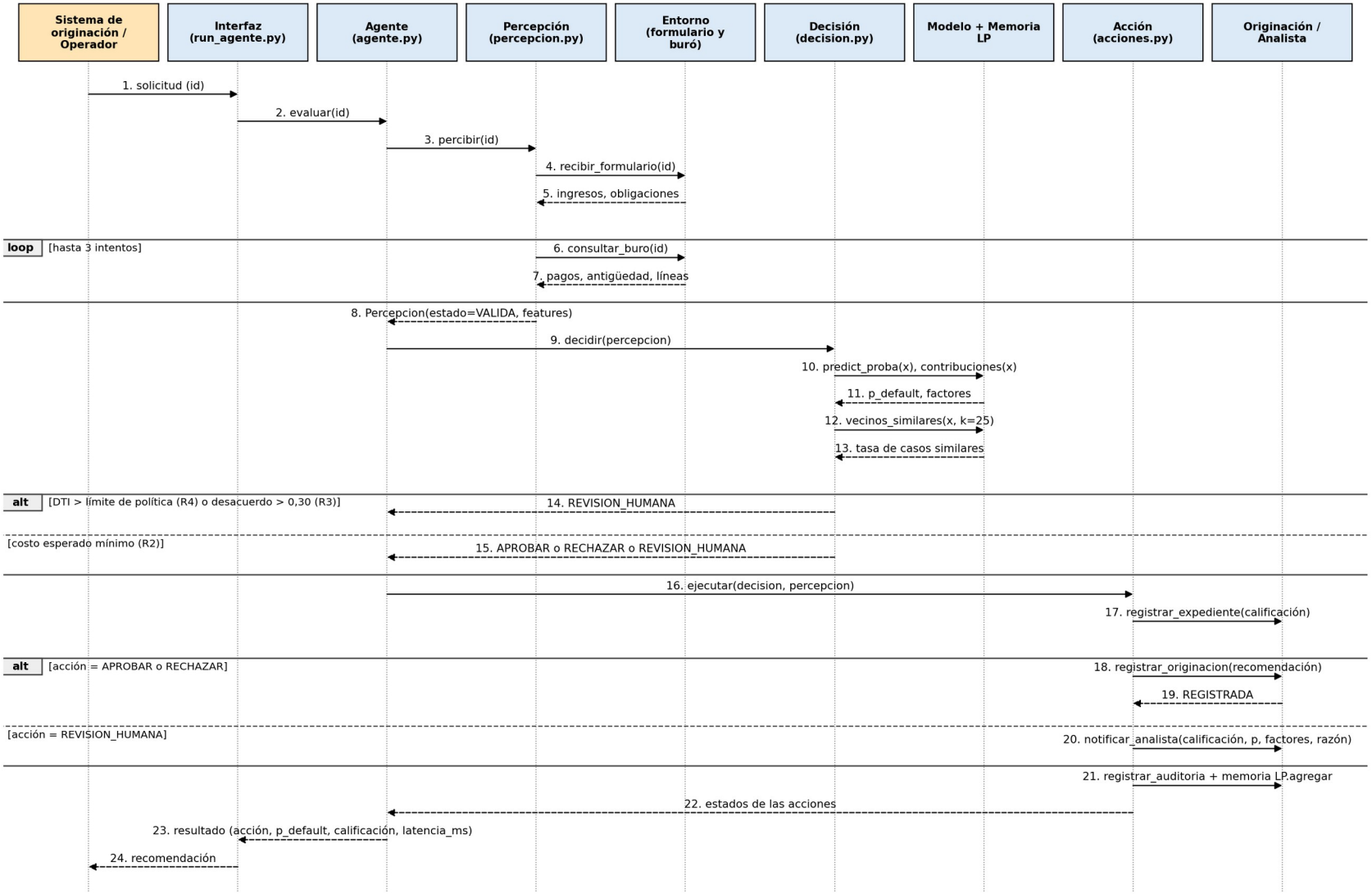


Figura 2. Escenario típico: evaluación normal de una solicitud (fuente: docs/diagramas/secuencia_tipico.mmd)

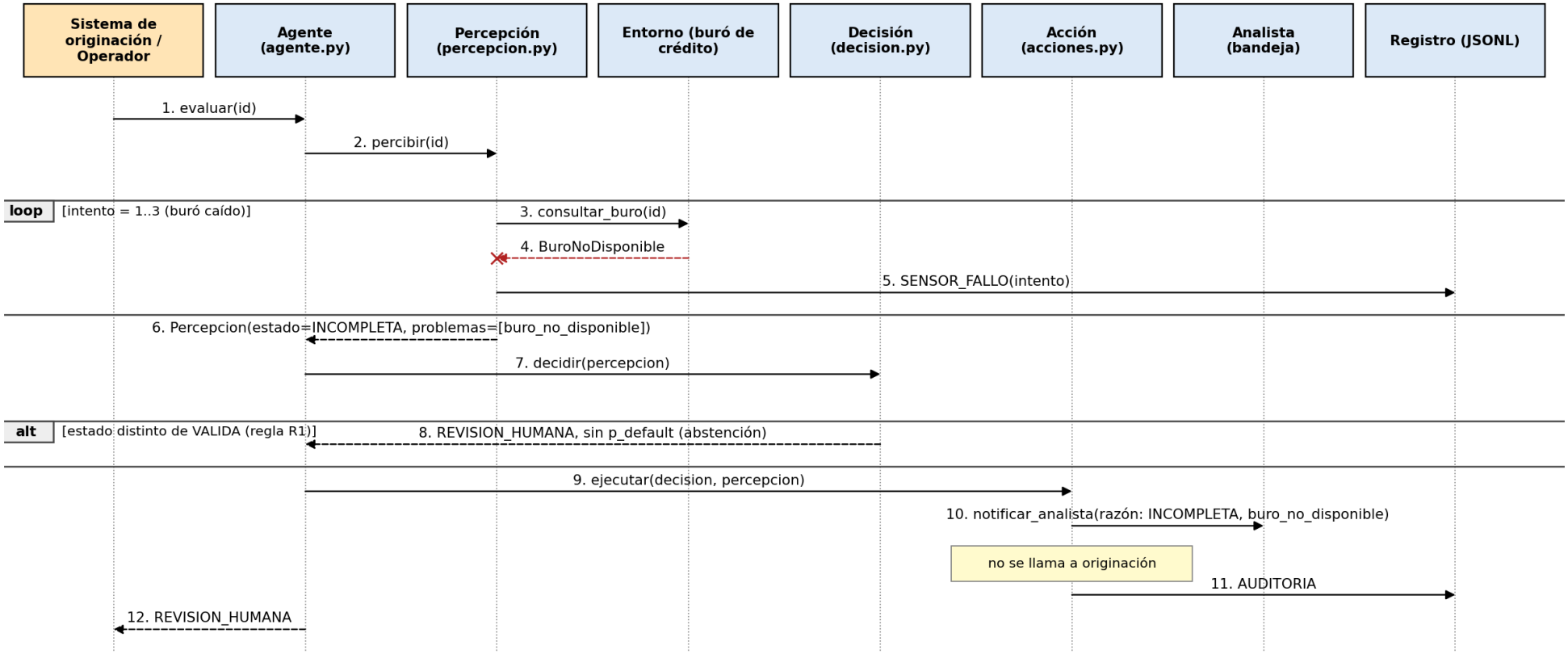


Figura 3. Escenario de excepción 1: buró de crédito caído, escenario E12 (fuente: secuencia_excepcion_buro.mmd)

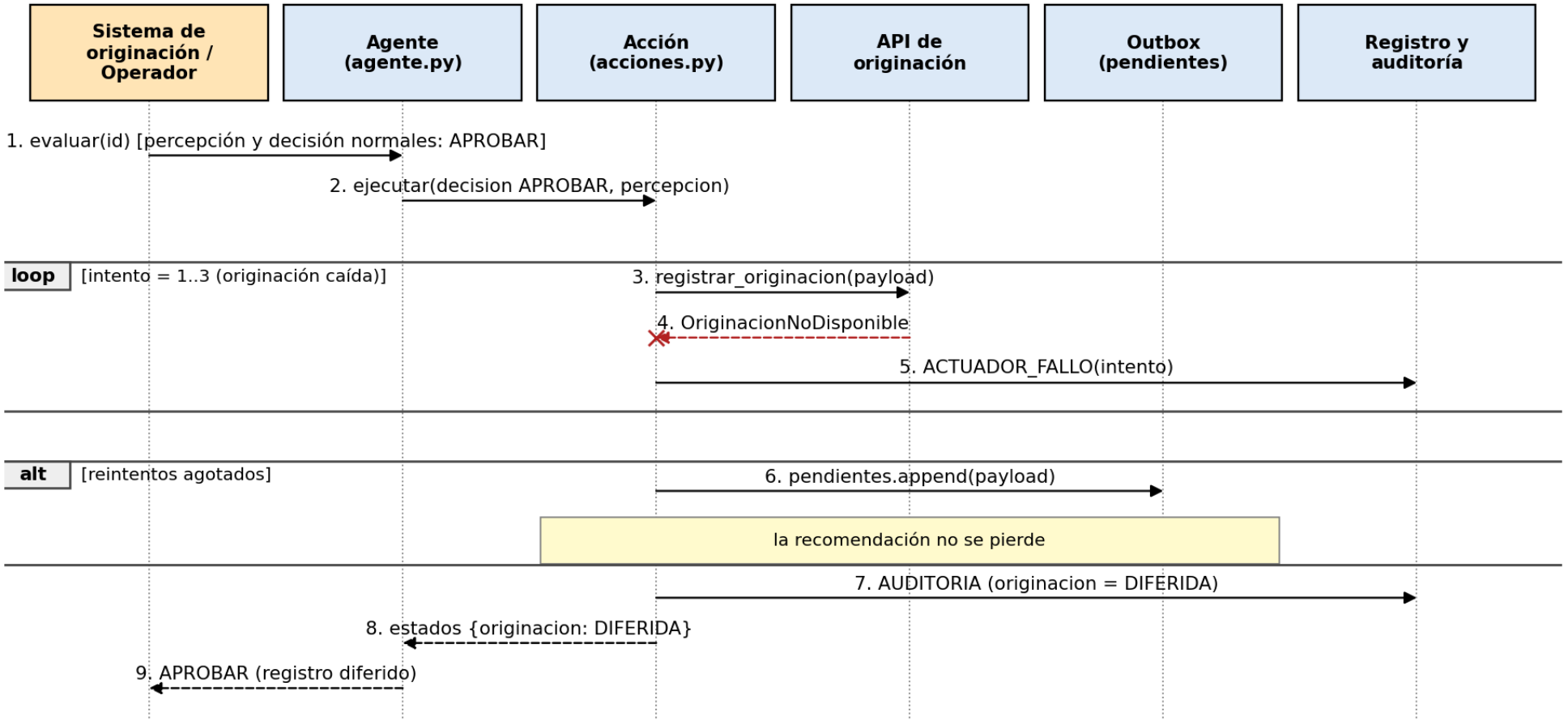


Figura 4. Escenario de excepción 2: sistema de originación caído, escenario E14 (fuente: secuencia_excepcion_originacion.mmd)

9.6 Diferencias entre diseño e implementación

Se comparó cada elemento del diseño (secciones 2 a 8) con el código. El tipo de agente, el ambiente y el ciclo coinciden; las diferencias encontradas se listan a continuación.

Tabla 8. Diferencias entre el diseño y la implementación

Elemento del diseño	Implementación realizada	Diferencia encontrada	Justificación	Impacto
Agente de aprendizaje con rasgos de utilidad	Regresión logística recalibrable y minimización del costo esperado	Ninguna (coincide; ver pruebas M7 y escenarios E01 a E03)	—	—
Ambiente parcialmente observable y estocástico	Simulador con ruido latente no observable por el agente	Ninguna	—	—
Historial de pagos: datos tabulares, categóricos y temporales	Dos variables numéricas: pagos atrasados en 24 meses y antigüedad	Simplificación	Tiempo disponible y ausencia de datos reales	Menor riqueza predictiva
Ingresos desde formulario, sistema financiero o API de verificación	Solo formulario simulado	Se omiten las otras fuentes	No hay acceso a servicios reales	Sin verificación cruzada de ingresos
DTI calculado a partir de obligaciones registradas	obligaciones_mensuales se agregó como entrada explícita	Entrada nueva no listada en la Tabla 1	Es necesaria para calcular el DTI	Ninguno
Historial de decisiones sobre solicitudes similares	Búsqueda de los 25 casos similares con resultado conocido	Se usan resultados, no las decisiones previas	El resultado es la señal que permite contrastar el modelo	Ninguno relevante
Abstención ante «incertidumbre elevada»	Regla R3: desacuerdo > 0,30 entre el modelo y los casos similares	Se concretó la incertidumbre como un desacuerdo medible	Debe ser operacionalizable	Umbral 0,30 sin calibración empírica
Políticas y umbrales de la entidad	Costos 4 / 1 / 0,5 y DTI máximo 0,80 en config.json	Valores definidos por los autores	No hay política institucional real	Cerca de la mitad de las solicitudes se remite a revisión
Resultado observado llega por seguimiento de cartera	El simulador lo entrega al instante	Sin retraso temporal	Simplificación de la simulación	No se prueba el manejo de resultados tardíos
Registro en BD de auditoría; API de originación; notificaciones	Listas en memoria y archivo JSONL; funciones del entorno simulado	Sin sistemas reales	Sin acceso a infraestructura	No se prueba integración real
Recalibración periódica	revisar_aprendizaje() invocable	No está programada	Alcance del curso	Requiere activación manual
Muestra de 5.000 solicitudes históricas reales	5.000 solicitudes sintéticas (3.500 entrenamiento y 1.500 prueba)	Datos sintéticos	No se dispone de datos reales	El AUC refleja la señal del simulador, no la de una entidad real
Meta: reducir 20 % el tiempo frente al proceso manual	Se mide la latencia p95 y la reducción frente a la línea base manual	Medida ejecutada	Línea base medida con el script del repositorio	Persona frente a agente en memoria
Entrada por formulario	app.py: interfaz web	No estaba en el diseño	Opción A	Ninguno
Métricas complementarias (precisión, recall, falsos positivos, subgrupos)	ECE con umbral; recall y tasas como información sin umbral	Subgrupos no evaluados	El simulador no tiene atributos de subgrupo	Equidad sin verificar

9.7 Pruebas de desempeño

Procedimiento. Los umbrales se fijaron en config/umbrales.json antes de ejecutar las pruebas (SHA-256 f67a40474eb00443…, registrado en logs/resultados_pruebas.json). Las pruebas con aleatoriedad se ejecutaron con 5 semillas (1 a 5) y se reportan media y desviación estándar. Los comandos para reproducirlas están en el README.

Tabla 9. Umbrales y su justificación

Medida	Justificación del umbral (fijado antes de ejecutar)
M1. AUC-ROC	0,75 es la meta del diseño y del caso de uso del catálogo.
M2. Latencia p95	200 ms es un tiempo interactivo razonable para una recomendación; complementa la reducción del 20 % (M2b), que compara contra la línea base manual.
M2b. Reducción del 20 %	Meta original del diseño; probada con la línea base manual.
M3. Remisión ante entradas inválidas	100 %: la validación es determinista y el diseño obliga a la abstención.
M4. ECE	0,05 es un límite habitual de buena calibración con 10 bins.
M5. Trazabilidad	100 %: el diseño exige registrar entradas, salida y versión de cada solicitud.
M6a. Desviación del AUC	0,02 (dos puntos de AUC) como variación aceptable entre muestras.
M6b. Reproducibilidad	100 %: con la misma semilla el agente debe ser idéntico.
M7a. Detección de deriva	100 %: el diseño exige detectar cambios en la distribución.
M7b. Mejora tras recalibrar	80 % de las semillas (4 de 5) como mejoría consistente.

Tabla de resultados obtenidos

Tabla 10. Resultados de las pruebas (fuente: logs/resultados_pruebas.csv; generado por scripts/run_pruebas_desempeno.py)

Medida de desempeño	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia (log, captura, minuto del video)
M1. Discriminación: AUC-ROC (Mann-Whitney) sobre solicitudes de prueba sintéticas	AUC = P(score_default > score_no_default); promedio de 5 semillas	>= 0,75	7500	0,7793 ± 0,0101 (min 0,7698)	Sí	logs/resultados_pruebas.json#M1; logs/pruebas_eventos.jsonl
M2. Tiempo de respuesta por evaluación (extremo a extremo, entorno simulado)	p95 de latencia_ms de EVALUACION_COMPLETA	<= 200 ms	7500	p95 = 0,701 ms; media = 0,489 ms	Sí	logs/resultados_pruebas.json#M2; logs/pruebas_eventos.jsonl
M2b. Reducción del 20 % del tiempo frente al proceso manual	(T_manual - T_agente) / T_manual	>= 20 %	10	T_manual = 3,6 s; T_agente = 0,489 ms; reducción = 99,99 %	Sí	config/linea_base_manual.json; logs/linea_base_manual_tiempos.csv
M3. Remisión correcta ante entradas inválidas, incompletas o fuera de dominio	casos con REVISION_HUMANA, sin p_default y sin llamada a originación / casos invalidos	>= 100 %	1000	1000/1000 = 100,0 %	Sí	logs/resultados_pruebas.json#M3; logs/pruebas_eventos.jsonl
M4. Calibración de probabilidades: ECE (10 bins)	ECE = Σ_b (n_b/n)·|frec_obs_b − prob_media_b|; promedio de 5 semillas	<= 0,05	7500	0,0237 ± 0,0066 (max 0,0328)	Sí	logs/resultados_pruebas.json#M4; logs/pruebas_eventos.jsonl
M5. Trazabilidad: evaluaciones con cadena completa de eventos y auditoría (releida del log)	evaluaciones completas / evaluaciones totales	>= 100 %	9000	9000/9000 = 100,0 %	Sí	logs/resultados_pruebas.json#M5; logs/pruebas_eventos.jsonl
M6a. Estabilidad entre ejecuciones: desviación estándar del AUC	s = desv. estándar muestral de AUC en 5 semillas	<= 0,02	5	s = 0,0101	Sí	logs/resultados_pruebas.json#M1
M6b. Reproducibilidad: la misma semilla produce las mismas decisiones	salidas idénticas (acción, p, calificación) / comparaciones	>= 100 %	1500	1500/1500 = 100,0 %	Sí	logs/resultados_pruebas.json#M6b
M7a. Detección de deriva (cambio de distribución económica)	semillas con deriva detectada / semillas	>= 100 %	5	5/5 = 100 %	Sí	logs/resultados_pruebas.json#M7
M7b. Mejora tras recalibración: Brier en solicitudes con deriva	semillas con Brier_después < Brier_antes / semillas	>= 80 %	5	5/5 = 100 %	Sí	logs/resultados_pruebas.json#M7

Evidencia de funcionamiento (Opción A): la pestaña «3. Pruebas» de la app desplegada ejecuta en vivo las pruebas unitarias, los 17 escenarios y las medidas M1–M7 y muestra esta misma tabla; los archivos de la columna final son el respaldo.

Escenarios de prueba deterministas (17 casos) y pruebas unitarias (20)

Tabla 11. Escenarios (fuente: logs/escenarios.csv y logs/escenarios_eventos.jsonl)

ID	Escenario	Categoría	Resultado esperado	¿Cumple?
E01	Normal: bajo riesgo	normal	APROBAR; originacion REGISTRADA	Sí
E02	Normal: alto riesgo	normal	RECHAZAR; originacion REGISTRADA	Sí
E03	Normal: riesgo intermedio	normal	REVISION_HUMANA; caso en bandeja del analista	Sí
E04	Limite: DTI = 0,80 exacto (no excede la politica)	limite	estado VALIDA y sin regla R4	Sí
E05	Limite: DTI = 0,8001 (excede la politica)	limite	REVISION_HUMANA por R4	Sí
E06	Limite: 24 pagos atrasados (maximo permitido)	limite	estado VALIDA	Sí
E07	Invalida: 25 pagos atrasados (fuera de rango)	invalida	REVISION_HUMANA; sin llamada a originacion	Sí
E08	Invalida: ingreso faltante	invalida	REVISION_HUMANA; estado INCOMPLETA	Sí
E09	Invalida: ingreso negativo	invalida	REVISION_HUMANA; estado INVALIDA	Sí
E10	Invalida: ingreso fuera de dominio (9e9)	invalida	REVISION_HUMANA; estado FUERA_DOMINIO	Sí
E11	Invalida: valor no numerico ('abc')	invalida	REVISION_HUMANA; estado INVALIDA	Sí
E12	Fallo: buro de credito caido (sensor sin dato)	fallo	3 reintentos, REVISION_HUMANA por buro_no_disponible	Sí
E13	Adverso: buro falla 2 veces y responde al 3.er intento	adverso	2 SENSOR_FALLO y decision normal APROBAR	Sí
E14	Fallo: originacion caida	fallo	APROBAR; originacion DIFERIDA, recomendacion conservada en outbox y auditada	Sí
E15	Adverso: originacion falla 1 vez y se recupera	adverso	originacion REGISTRADA tras reintento	Sí
E16	Fallo: solicitud inexistente	fallo	sin excepcion; REVISION_HUMANA INVALIDA	Sí
E17	Repetibilidad: mismo caso 5 veces	repeticion	5 salidas identicas	Sí

Las 17 de 17 pruebas de escenario cumplen lo esperado. Los 17 escenarios incluyen casos normales, de límite (DTI igual y superior a la política; 24 y 25 pagos atrasados), entradas inválidas, fallos esperados (buró caído, originación caída, solicitud inexistente) y condiciones adversas (fallos transitorios con recuperación); E17 repite el mismo caso 5 veces. Las 20 pruebas unitarias (logs/unittest.txt) pasan. Aparte, M3 ejecutó 1.000 entradas corrompidas de 10 tipos (100 por tipo) y M5 incluyó 500 evaluaciones con fallos aleatorios del 30 % en buró y originación.

Métricas informativas (sin umbral): % de solicitudes aprobadas 37 %–42 %, rechazadas 10 %–11 %, remitidas a revisión 48 %–53 %; tasa de incumplimiento entre aprobadas 5,8 %–8,2 % frente a 22,3 %–24,2 % en la muestra; recall del incumplimiento (incumplidoras no aprobadas) 85,6 %–90,5 %.

Análisis de resultados

¿Qué se cumplió? Las nueve medidas ejecutables cumplen su umbral: AUC medio de 0,7793 (mínimo 0,7698), p95 de latencia de 0,701 ms, remisión correcta de 1.000 de 1.000 entradas inválidas, ECE medio bajo 0,05, trazabilidad completa en 9.000 evaluaciones, reproducibilidad exacta y detección y mejora tras recalibración en 5 de 5 semillas. ¿Qué no se cumplió? Ninguna medida incumplió su umbral. M2b (reducción del 20 % del tiempo) también se cumplió: T_manual = 3,6 s; T_agente = 0,489 ms; reducción = 99,99 %.

Por qué estos resultados deben leerse con cautela (causas y evidencia):

- **M1:** el AUC sobre datos sintéticos refleja la señal que el propio simulador contiene; demuestra que la tubería funciona, no que el agente discrimine bien en una entidad real. Además, el margen es estrecho: la peor semilla obtuvo 0,7698 frente a un umbral de 0,75 (logs/resultados_pruebas.json#M1).
- **M2:** el p95 es inferior a 1 ms en un entorno simulado en memoria, sin red ni consulta real al buró. La reducción de M2b (T_manual = 3,6 s; T_agente = 0,489 ms; reducción = 99,99 %) compara a una persona que decide sobre datos ya presentados con un programa en memoria; es una cota optimista y no incluye digitación, consulta real al buró ni verificación documental, por lo que no equivale al ahorro real en una entidad.
- **Carga de revisión:** el agente remite a revisión humana entre 48 % y 53 % de las solicitudes válidas. Es consecuencia del costo de revisión (0,5) y de los costos de error (4 y 1) fijados por los autores; si esos costos no reflejan a la entidad, la carga de los analistas sería mayor de lo deseable.

- **M7:** el PSI sobre las puntuaciones nunca superó su umbral de 0,2 (valores de 0,024 a 0,058); la deriva se detectó por la brecha de calibración (0,077 a 0,111 frente a 0,05). La deriva simulada cambia principalmente la tasa base de incumplimiento, que el PSI de las puntuaciones no captura bien. La recalibración mejoró el Brier en 5 de 5 semillas, pero la mejora es modesta (0,0056 a 0,0086), el ECE posterior sigue por encima de 0,05 en 2 de 5 semillas (hasta 0,080) y en la semilla 5 el AUC bajó de 0,7853 a 0,7823 (logs/resultados_pruebas.json#M7).

Cambios que podrían corregirlo: calcular el PSI por variable de entrada y monitorear la tasa de incumplimiento observada; recalibrar la probabilidad (por ejemplo, ajuste del intercepto o regresión isotónica) con más retroalimentación antes de reentrenar todo el modelo; calibrar los costos de decisión con datos de la entidad; repetir M2b con analistas reales; y repetir M1 con datos reales o con un conjunto público de crédito. Los umbrales no se modificaron después de observar los resultados.

9.8 Limitaciones y trabajo futuro

Limitaciones reales:

- Datos: todas las solicitudes son sintéticas y la señal predictiva la define el simulador; no hay evidencia de desempeño con datos reales ni de equidad entre subgrupos.
- Entorno: buró, originación y cartera son simulados; el resultado observado llega al instante, no meses después.
- Modelo: regresión logística con cinco variables y sin interacciones; la regla R3 usa un umbral (0,30) sin calibración empírica.
- Pruebas: la línea base manual se midió sobre solicitudes simuladas ya presentadas, sin digitación ni consulta real al buró; la latencia del agente no incluye redes ni personas.
- Aprendizaje: la revisión de deriva debe invocarse manualmente y su detección por PSI resultó poco sensible al tipo de deriva simulada.
- Decisión: los costos de error y de revisión son supuestos de los autores.

Mejoras propuestas, por prioridad:

- Alta: repetir M2b con analistas reales y datos de una entidad (tiempo con digitación y consulta real al buró).
- Alta: repetir M1 y M4 con un conjunto de datos público de riesgo crediticio o con datos reales anonimizados.
- Alta: añadir evaluación de equidad por subgrupos y revisión de variables sensibles.
- Media: PSI por variable y monitoreo de la tasa de incumplimiento; recalibración isotónica o por intercepto.
- Media: calibrar costos de decisión y el umbral de R3 con datos de la entidad.
- Baja: persistir la memoria de largo plazo entre ejecuciones, programar la revisión de aprendizaje.

9.9 Enlaces

Tabla 12. Enlaces de entrega

Recurso	Estado	Cómo obtenerlo
Repositorio (GitHub)	Publicado (comprobado por el integrante: abre sin iniciar sesión)	https://github.com/stevencontre/creditscore-agent
Plataforma de despliegue (Opción A: Streamlit Community Cloud)	Publicada; disponible hasta la fecha de calificación	https://creditscore-agent-7pyg7dzx7yiykdggfwoinw.streamlit.app/
Instrucciones de uso y casos de prueba sugeridos	En el repositorio	docs/guia_despliegue.md y pestaña «Evaluar una solicitud» de la app
Video de la demostración	No aplica: se eligió la Opción A (plataforma de despliegue)	—

Declaración de uso de herramientas de IA. El código, los diagramas y este documento se prepararon con ayuda de un asistente de IA (Claude, de Anthropic). Los resultados numéricos provienen de ejecuciones reales del código incluido. La bitácora docs/bitacora_uso_ia.md registra el uso y la verificación realizada por los integrantes, que deben revisar y poder explicar el código y el documento.

10. Conclusiones

El diseño del CREDITSCORE-AGENT se implementó como un agente de aprendizaje con decisión por utilidad, con un ciclo Percepción → Decisión → Acción identificable en el código, memoria de corto y largo plazo, abstención ante datos inválidos o incertidumbre, actuadores con manejo de fallos y registro con marca de tiempo. Las pruebas ejecutadas muestran que el agente cumple los umbrales definidos para discriminación, calibración, tiempo de respuesta, tolerancia a entradas inválidas, trazabilidad, reproducibilidad y adaptación a la deriva, siempre sobre datos sintéticos.

Estos resultados no demuestran todavía que el agente sea apto para una entidad financiera. Faltan la validación con datos reales, la evaluación de equidad por subgrupos y la calibración de los costos de decisión con la política de una entidad. La decisión de implementar este agente frente a Cartrescue-agent se apoyó en puntajes estimados para este último y debe confirmarse con su diseño real. Por el impacto de las decisiones crediticias, el agente debe seguir diseñándose como apoyo a la decisión, con revisión humana, auditoría y control de sesgos.

Referencias

Curso de Inteligencia Artificial. (2026). Intelligent Agents: Use Cases [Documento de casos de uso]. Material proporcionado en la asignatura.

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357–362. https://doi.org/10.1038/s41586-020-2649-2

IBM. (2025, 10 de marzo). What are the components of AI agents? [Artículo de C. Stryker]. IBM Think. https://www.ibm.com/think/topics/components-of-ai-agents

Mota, J., y Moncayo, C. (2023). Un análisis del perfil de riesgo en la dinámica de préstamos persona a persona mediante clústeres de K-medias. Economía: Teoría y Práctica, 31(59), 119–138.

Nortey, E. N. N., Asante-Koranteng, J., Atemkeng, M., Ansah-Narh, T., Mensah, D., Davis, R., y Laryea, R. A. (2026). An Optimised Greedy-Weighted Ensemble Framework for Financial Loan Default Prediction. arXiv:2603.18927. https://arxiv.org/abs/2603.18927

Russell, S. J., y Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4.ª ed. global). Pearson. https://aima.cs.berkeley.edu/globalindex.html